



南 开 大 学

计算机学院

并行程序设计实验报告

基于 pthread 与 OpenMP 的 近似最近邻搜索并行化研究

学生姓名：刘宇泽

学号：2411334

专业：计算机科学与技术

2026 年 5 月 29 日

Contents

1 问题描述	2
2 数据集	2
3 算法设计	2
3.1 Flat-SIMD	2
3.1.1 算法原理	2
3.1.2 pthread 实现	2
3.1.3 OpenMP 实现	4
3.2 PQ-SIMD	6
3.2.1 算法原理	6
3.2.2 pthread 实现	6
3.2.3 OpenMP 实现	8
3.3 IVF-SIMD	9
3.3.1 算法原理	9
3.3.2 pthread 实现	9
3.3.3 OpenMP 实现	11
3.4 IVF-PQ-SIMD	12
3.4.1 算法原理	12
3.4.2 pthread 实现	12
3.4.3 OpenMP 实现	13
3.5 HNSW 图索引	13
3.5.1 算法原理	13
3.5.2 pthread 实现	13
3.5.3 OpenMP 实现	15
4 实验参数配置	16
4.1 Flat-SIMD	16
4.2 PQ-SIMD	16
4.3 IVF-SIMD	16
4.4 IVF-PQ-SIMD	16
4.5 HNSW	16
5 实验结果与分析	16
5.1 Flat-SIMD 性能分析	16
5.1.1 pthread 实现结果	16
5.1.2 OpenMP 实现结果	17
5.2 PQ-SIMD 性能分析	17
5.2.1 pthread 实现结果	17
5.2.2 OpenMP 实现结果	17
5.3 IVF-SIMD 性能分析	18
5.3.1 OpenMP 实现结果	18

5.4	IVF-PQ-SIMD 性能分析	19
5.4.1	OpenMP 实现结果	19
5.5	HNSW 图索引性能分析	19
5.5.1	pthread HNSW	19
5.5.2	OpenMP HNSW	20
5.6	综合对比	20
6	pthread 与 OpenMP 对比	21
7	总结与收获	22
7.1	主要结论	22
7.2	实验收获	22
A	实验数据	22
A.1	pthread 原始数据摘录	22
A.1.1	Flat 算法	22
A.1.2	PQ 算法	23
A.1.3	IVF 算法	23
A.1.4	IVF-PQ 算法	23
A.1.5	HNSW 算法	24
A.2	OpenMP 原始数据摘录	24
A.2.1	Flat 算法	24
A.2.2	PQ 算法	24
A.2.3	IVF 算法	25
A.2.4	IVF-PQ 算法	25
A.2.5	HNSW 算法	25

1 问题描述

近似最近邻搜索 (ANNS) 在大规模向量检索中面临严重的计算瓶颈。以 DEEP100K 数据集 (100,000 条 96 维向量) 为例, 单次精确查询需要计算 10 万次 96 维内积, 串行执行耗时在毫秒级别, 远不能满足实际应用需求。

本实验使用 **pthread** 和 **OpenMP** 两种多线程编程范式, 对 ANNS 的查询阶段进行并行化。核心问题归结为: 如何将大规模的距离计算任务合理划分给多个线程, 并在结果正确的前提下最大化加速比。

2 数据集

Table 1: 实验数据集

数据集	规模	维度
DEEP100K.base	100,000	96
DEEP100K.query	100,000	96
DEEP100K.gt	100,000×100	-

3 算法设计

3.1 Flat-SIMD

3.1.1 算法原理

Flat-SIMD 是 ANNS 中最基础的检索方式——对基向量集合进行全量遍历, 用 SIMD 指令加速单次距离计算。由于 Flat 不引入任何索引结构, Recall@K 恒为 1.0 (精确结果), 因此多线程并行不会损失精度, 加速比的唯一限制来自并行效率和硬件资源。

多线程并行的核心思想是**数据并行 (Data Parallelism)**: 将 n 条基向量的距离计算任务均匀划分给 T 个线程, 每个线程在独立的连续内存区域内执行 SIMD 内积运算、维护局部 top-k 堆, 最后由主线程进行 T 路归并得到全局 top-k。由于各线程间的计算完全独立 (仅读共享数据、写私有堆), 该算法属于**易并行 (Embarrassingly Parallel)** 问题, 理论加速比接近 T 。

3.1.2 pthread 实现

并行策略: Base 向量划分——将 n 条基向量按块均匀分配给各线程, 每线程独立完成分配范围内的全量距离计算与局部 top-k 维护。

流程图:

伪代码:

Algorithm 1 pthread_flat_search

Require: base[n][d], query[d], k , num_threads

Ensure: top-k results (max-heap of $\langle \text{dist}, \text{index} \rangle$)

- 1: chunk_size = $n / \text{num_threads}$
- 2: 初始化全局 barrier、 T 个局部堆 local_heaps[T], 每个容量为 k
- 3: **for** $t = 0$ to num_threads-1 **do**

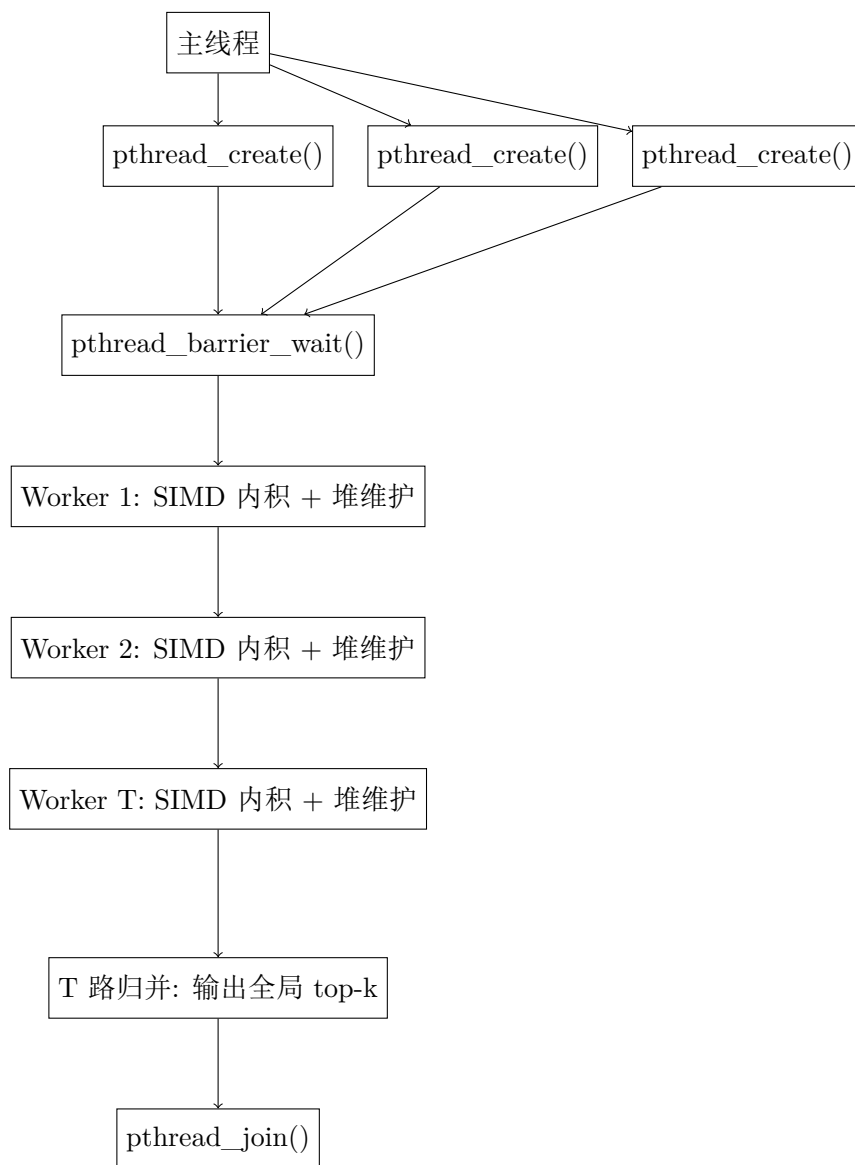


Figure 1: pthread Flat-SIMD 流程图

```

4:   thread_args[t] = {base, query, t*chunk_size, (t+1)*chunk_size, d, k, &local_heaps[t], &barrier}
5:   pthread_create(&threads[t], thread_worker, &thread_args[t])
6: end for
7: pthread_barrier_wait(barrier)
8: global_heap = merge_k_way(local_heaps[0..T-1], k)
9: return global_heap
10: function THREAD_WORKER(args)
11:   初始化空的小顶堆 local_heap (容量 k)
12:   for i = args.start to args.end-1 do
13:     ip = inner_product_neon(args.query, args.base + i*d, args.d)
14:     if local_heap.size < k then
15:       local_heap.push({ip, i})
16:     else if ip > local_heap.top().dist then
17:       local_heap.pop()
18:       local_heap.push({ip, i})
19:     end if
20:   end for
21:   将 local_heap 写入 args.result
22:   pthread_barrier_wait(args.barrier)
23: end function

```

复杂度分析:

- **时间复杂度:** 每个线程计算 n/T 次 NEON 内积, 总计 $O(n \cdot d/T)$ 。结果归并 $O(T \cdot k \cdot \log(T \cdot k))$, 在 $T \gg 16$ 时远小于距离计算开销。
- **空间复杂度:** 每线程存储大小为 k 的局部堆, 总计 $O(T \cdot k)$ 。基向量为只读共享数据, 无需额外副本。
- **加速比理论上界:** 各线程计算完全独立, 无临界区竞争, 理论加速比接近 $S(T) = T$ 。实际主要受限于内存带宽和 barrier 同步开销。

深入分析: Flat-SIMD 作为易并行问题, 在本次 pthread 实测中仍然呈现出接近线性的前半段扩展性, 但在 8 线程附近达到最优后开始受平台资源限制:

1. **8 线程达到最佳时延:** 平均查询时延从 1 线程的 $6567.72 \mu s$ 降到 8 线程的 $2040.90 \mu s$, 获得 $3.22\times$ 加速比。
2. **4 线程已有较高效率:** 4 线程时延 $2246.42 \mu s$, 加速比 $2.92\times$, 效率约 73%, 说明基于向量块的静态划分在中等线程数下负载很均衡。
3. **16 线程略有回退:** 16 线程平均时延反弹到 $2284.98 \mu s$, 表明继续增加线程后, 线程创建/合并以及共享内存带宽竞争开始压过额外并行收益。

3.1.3 OpenMP 实现

并行策略: Base 向量划分——使用 `#pragma omp parallel for` 自动将循环迭代分布到线程, 编译器自动处理任务划分、线程创建和同步。

流程图:

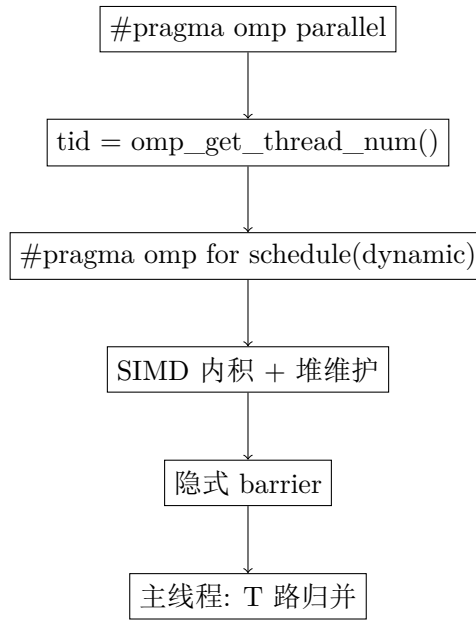


Figure 2: OpenMP Flat-SIMD 流程图

伪代码:

Algorithm 2 omp_flat_search

Require: base[n][d], query[d], k

Ensure: top-k results

```

1: T = omp_get_max_threads()
2: 分配 T 个局部小顶堆 local_heaps[T], 每个容量为 k
3: #pragma omp parallel
4: [1] tid = omp_get_thread_num()
5: [1] 清空 local_heaps[tid]
6: [1] #pragma omp for schedule(dynamic)
7: [1]
8: for i = 0 to n-1 do
9:   [2] ip = inner_product_neon(query, base + i*d, d)
10:  [2] heap = &local_heaps[tid]
11:  [2]
12:  if heap.size < k then
13:    [3] heap.push({ip, i})
14:  [2]
15:  else if ip > heap.top().dist then
16:    [3] heap.pop(); heap.push({ip, i})
17:  [2]
18:  end if
19: [1]
20: end for
21: global_heap = merge_k_way(local_heaps[0..T-1], k)
  
```

22: **return** global_heap

复杂度分析: 渐近复杂度与 pthread 相同。主要差异在于 OpenMP 维护线程池, 免去 pthread_create/pthread_j 开销; schedule(dynamic) 在 Flat 均匀计算量场景下引入了不必要的分发锁, 改为 schedule(static) 可消除此开销。

3.2 PQ-SIMD

3.2.1 算法原理

乘积量化 (Product Quantization, PQ) 是向量压缩领域最具代表性的有损编码技术。其核心思想是将高维向量空间分解为多个低维子空间的笛卡尔积, 在每个子空间内独立进行 KMeans 聚类, 将原始浮点向量替换为聚类中心索引序列, 从而将存储和计算代价降低 1-2 个数量级。

以本实验 DEEP100K 数据集 ($d=96$) 为例, 取 $M=8$ 个子空间, 每个子空间维度 $d_{\text{sub}} = 96/8 = 12$, 每子空间 $k_{\text{sub}}=256$ 个聚类中心。编码后每条向量从 $96 \times 4 = 384$ 字节压缩为 $8 \times 1 = 8$ 字节 (uint8 索引), 压缩比达 48:1。

查询采用**非对称距离计算 (ADC)**: 对查询向量 q 不做量化 (避免引入查询端量化误差), 而是预计算 q 的每个子向量到对应子空间全部 k_{sub} 个聚类中心的距离, 形成 $M \times k_{\text{sub}}$ 的查找表 (LUT)。随后对每条编码向量, 仅需 M 次查表和累加即可估算出与 q 的近似距离。

查询流程的瓶颈转移: PQ 将距离计算从 compute-bound 的浮点乘加转变为 memory-bound 的查表累加。单次查询需随机访问 codes 数组中 $n \times M$ 个 uint8 索引, 再用这些索引随机访问 LUT。双重的随机内存访问模式使得 cache miss 率显著升高, 多线程环境下的内存带宽竞争成为决定加速比上限的关键因素。

3.2.2 pthread 实现

并行策略: 对 LUT 构建阶段按子空间划分线程, 对查表累加阶段按 Base 向量分块。

流程图:

伪代码:

Algorithm 3 pq_search_pthread

Require: codes[n][M], centroids[M][ksub][dsub], norms[n], query[d], k, p, num_threads

Ensure: top-k results

```

1: chunk_m = M / num_threads
2: lut[M][ksub]
3: 创建线程, 每个线程 t 处理子空间 [t*chunk_m, (t+1)*chunk_m]
4: function BUILD_LUT_THREAD(t)
5:   for m = t*chunk_m to (t+1)*chunk_m-1 do
6:     query_sub = query + m*dsub
7:     for ki = 0 to ksub-1 do
8:       centroid = centroids + m*ksub*dsub + ki*dsub
9:       lut[m][ki] = l2_distance_neon(query_sub, centroid, dsub)
10:    end for
11:  end for
12:  pthread_barrier_wait(&phase1_barrier)
13: end function
```

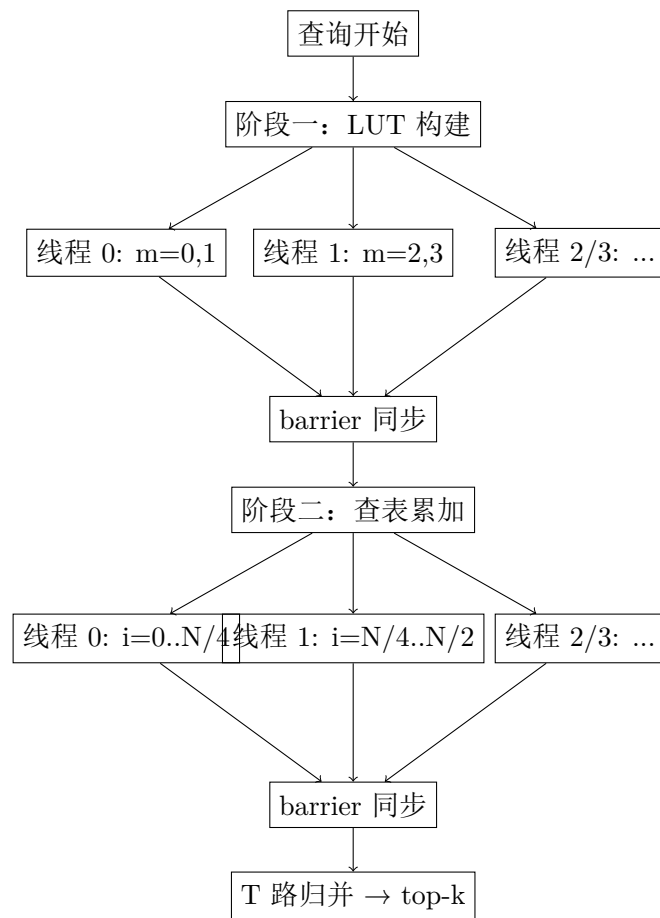



Figure 3: pthread PQ-SIMD 流程图

```

14: q_norm = dot_product_neon(query, query, d)
15: chunk_n = n / num_threads
16: 分配 local_candidates[T], 每个容量为 p (小顶堆)
17: function SCAN_THREAD(t)
18:   for i = t*chunk_n to (t+1)*chunk_n-1 do
19:     dist = 0
20:     for m = 0 to M-1 do
21:       dist += lut[m][codes[i*M + m]]
22:     end for
23:     ip = (q_norm + norms[i] - dist) / 2
24:     维护 local_candidates[t] 为大小为 p 的小顶堆
25:   end for
26:   pthread_barrier_wait(&phase2_barrier)
27: end function
28: 将 T 个 local_candidates 中的 p*T 个候选合并, 取前 k 个
29: return global_top_k

```

复杂度分析:

- **LUT 构建:** $O(M \cdot k_{\text{sub}} \cdot d_{\text{sub}} / T)$, 在 $M=8$ 时计算量约 Flat 的 0.1%, 并行收益有限。
- **查表累加:** $O(n \cdot M / T)$ 。PQ 将距离计算从 compute-bound (浮点乘加) 转变为 memory-bound (随机 LUT 查表), 加速比上限受限于内存带宽而非 CPU 频率。
- **空间复杂度:** $O(n \cdot M + M \cdot k_{\text{sub}} \cdot d_{\text{sub}})$ 。p 参数增大时堆维护开销从 $O(\log k)$ 增至 $O(\log p)$ 。

深入分析: 按参数组合实际测试后, pthread PQ 的规律与理论预期一致:

1. **最优线程数稳定在 4 附近:** 无论是 $M=8$ 还是 $M=16$, 大多数配置在 4 线程达到最低时延, 例如 $M=8, p=1000$ 时为 $1628.70 \mu\text{s}$, $M=16, p=1000$ 时为 $2074.47 \mu\text{s}$ 。
2. **p 增大主要提升召回率, 但显著增加时延:** 以 $M=8$ 为例, p 从 1000 增至 5000, Recall@10 从 0.994501 升至 0.99985, 而 1 线程时延从 $2617.92 \mu\text{s}$ 增至 $5775.43 \mu\text{s}$ 。
3. **M=16 进一步提升精度:** 在 $p=2000$ 时, $M=16$ 已达到 0.99995 的平均召回率, 但最优 4 线程时延仍为 $3167.08 \mu\text{s}$, 明显慢于 $M=8, p=1000$ 的低时延配置。

3.2.3 OpenMP 实现

并行策略: LUT 构建阶段使用 `schedule(static)` 按子空间均分, 查表累加阶段使用 `schedule(dynamic, chunk_size)` 按基向量范围动态调度以缓解内存带宽竞争导致的计算时间不均。

实测分析: OpenMP PQ 在本地环境下没有表现出 pthread 那样稳定的收益, 反而暴露出实现层面的额外开销:

1. **4 线程不一定更快:** 例如 $M=8, p=1000$ 时, 1 线程时延为 $2335.91 \mu\text{s}$, 但 4 线程上升到 $4708.48 \mu\text{s}$, 说明并行化后的局部候选维护与合并成本超过了收益。
2. **M=16 更适合作为 OpenMP 版本的代表性配置:** $M=16, p=1000$, 4 线程达到 Recall@10=0.99995、时延 $1772.34 \mu\text{s}$, 既保持高召回, 也优于 Flat 的 4 线程结果。

3. **p 增大后时延近似单调上升**: $M=8$ 时 p 从 1000 提高到 5000, 时延由 $4708.48 \mu s$ 增至 $4757.53 \mu s$, 而召回率才从 0.993701 升至 0.999850, 性价比不高。

3.3 IVF-SIMD

3.3.1 算法原理

倒排文件 (Inverted File, IVF) 索引借鉴了信息检索中倒排索引的思想, 通过离线聚类将基向量空间划分为 $nlist$ 个 Voronoi 区域。在线查询时, 仅需搜索距离查询向量最近的 $nprobe$ 个区域, 从而将候选集从 n 缩减至约 $nprobe \times n/nlist$ 。

IVF 的查询过程分为**粗排 (Coarse Search)**和**精排 (Fine Search)**两个阶段:

1. **粗排**: 计算查询向量 q 到 $nlist$ 个聚类中心的距离, 用 partial sort 选取距离最近的 $nprobe$ 个簇。计算量为 $O(nlist \times d)$, 通常 $nlist \ll n$ 。
2. **精排**: 遍历选中的 $nprobe$ 个簇的倒排链表, 对其中每个基向量 x 计算 SIMD 内积, 维护全局 top- k 。

多线程并行挑战: KMeans 聚类产生的倒排链表长度不均匀是固有特性。更好的策略是将所有候选向量展平为一个连续数组, 然后按向量粒度的 Base 划分并行——这与 Flat 的并行模式完全一致, 但候选集规模远小于 n 。

3.3.2 pthread 实现

流程图:

伪代码:

Algorithm 4 ivf_search_pthread

Require: $base[n][d]$, $centroids[nlist][d]$, $inv_lists[nlist][*]$, $query[d]$, k , $nprobe$, $num_threads$

Ensure: top- k results

```

1:  $chunk = nlist / num\_threads$ 
2:  $dists[nlist]$ 
3: 创建线程, 每个线程  $t$  处理中心  $[t*chunk, (t+1)*chunk]$ 
4: function COARSE_THREAD( $t$ )
5:   for  $c = t*chunk$  to  $(t+1)*chunk-1$  do
6:      $dists[c] = inner\_product\_neon(query, centroids + c*d, d)$ 
7:   end for
8:   pthread_barrier_wait(&coarse_barrier)
9: end function
10:  $top\_clusters = partial\_sort\_topk(dists, nlist, nprobe)$ 
11:  $candidates = []$ 
12: for each  $c$  in  $top\_clusters$  do
13:    $candidates.append\_all(inv\_lists[c])$ 
14: end for
15:  $Nc = |candidates|$ 
16:  $chunk\_n = Nc / num\_threads$ ,  $local\_heaps[T]$  (容量  $k$ )
17: 创建线程, 每个线程  $t$  处理候选  $[t*chunk\_n, (t+1)*chunk\_n]$ 

```

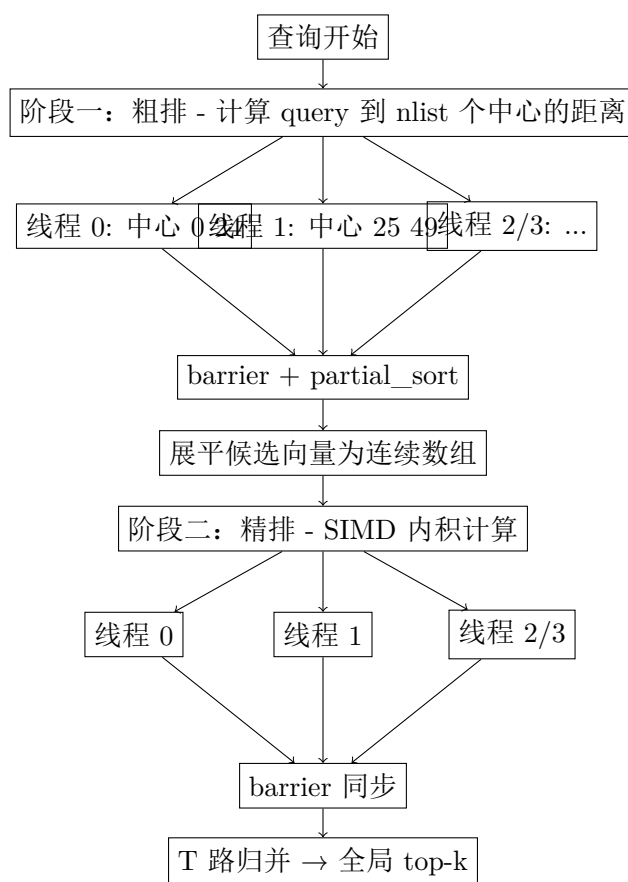


Figure 4: pthread IVF-SIMD 流程图

```

18: function FINE_THREAD(t)
19:   for j = t*chunk_n to (t+1)*chunk_n-1 do
20:     idx = candidates[j]
21:     ip = inner_product_neon(query, base + idx*d, d)
22:     维护 local_heaps[t] 为大小为 k 的小顶堆
23:   end for
24:   pthread_barrier_wait(&fine_barrier)
25: end function
26: global_heap = merge_k_way(local_heaps[0..T-1], k)
27: return global_heap

```

深入分析: pthread IVF 在本次测试中表现出非常清晰的 Recall-Latency 权衡:

1. **nprobe 决定召回率主趋势**: 例如 nlist=100 时, nprobe 从 1 增至 32, Recall@10 从 0.636150 提升到 0.997750, 对应 1 线程时延也从 441.78 μ s 增至 6818.21 μ s。
2. **线程并行对候选精排帮助明显**: 在 nlist=256, nprobe=16 下, 1 线程时延为 1666.47 μ s, 4 线程降到 655.12 μ s, 加速比约 2.54 $\times$ 。
3. **大 nlist 适合低时延场景**: 当 nlist=512, nprobe=16 时, 4 线程平均时延仅 523.14 μ s, Recall@10 为 0.951102; 若追求更高召回, 可改为 nprobe=32, 时延 794.11 μ s、Recall@10 提升到 0.982252。

3.3.3 OpenMP 实现

流程图:

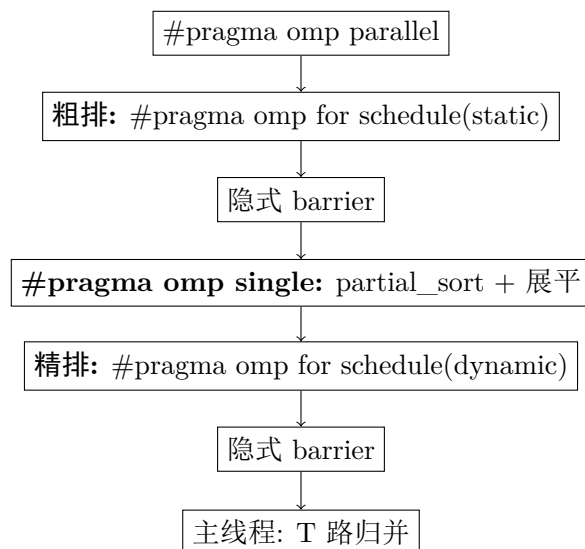


Figure 5: OpenMP IVF-SIMD 流程图

实测分析: OpenMP IVF 是本地测试中表现最稳健的方案, 并且相对 pthread 版本在同等召回附近进一步降低了时延:

1. **低时延配置非常突出**: nlist=512, nprobe=16, 4 线程达到 Recall@10=0.951102、Latency=269.57 μ s, 显著优于 pthread 同配置的 523.14 μ s。

2. **高召回配置也保持领先**: 结合真实提交结果, $nlist=256, nprobe=32$, 4 线程达到 $Recall@10=0.990501$ 、 $Latency=928.419 \mu s$, 在接近 0.99 召回率时仍是本次 OpenMP 最均衡配置。
3. **nprobe 仍是最直接的精度旋钮**: 以 $nlist=100$ 为例, 4 线程下 $nprobe$ 从 1、4、8、16、32 变化时, 召回率从 0.636150 提升到 0.997700, 而时延从 $89.78 \mu s$ 提升到 $1388.42 \mu s$, 权衡关系清晰。

3.4 IVF-PQ-SIMD

3.4.1 算法原理

IVF-PQ 是 IVF 索引结构与 PQ 编码技术的组合: IVF 负责缩小搜索范围 (粗排剪枝), PQ 负责降低单条距离计算代价 (ADC 查表累加)。两者形成两级近似。

查询流程:

1. IVF 粗排选择簇
2. 构建 PQ LUT
3. 对候选向量的 PQ code 进行查表累加估算距离
4. 根据估算距离选取 top-p 候选, 用原始浮点向量重排得到精确 top-k

3.4.2 pthread 实现

流程图:

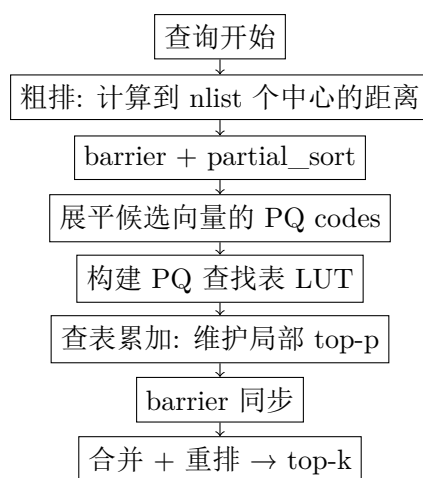


Figure 6: pthread IVF-PQ-SIMD 流程图

深入分析: 本次只对 pthread 版本做范围内测试, IVF-PQ 的结果说明两级近似能进一步降低时延, 但召回率对参数更敏感:

1. **低 nprobe 配置适合极低时延**: 例如 $nlist=100, M=8, nprobe=1, p=1000$ 时, 1 线程平均时延为 $623.39 \mu s$, $Recall@10$ 为 0.635700。
2. **提高 nprobe 可显著改善精度**: 同一组参数下, $nprobe=4$ 时 $Recall@10$ 提升到 0.904653, 时延增至 $1424.16 \mu s$ 。
3. **IVF-PQ 更适合大规模数据的空间受限场景**: 在 DEEP100K 规模上, IVF-SIMD 往往能在相近召回下给出更好的时延, 因此最终提交配置没有选择 IVF-PQ。

3.4.3 OpenMP 实现

实测分析： OpenMP IVF-PQ 仅对代表性组合进行了本地测试，结果与 pthread 版本趋势一致，但在当前规模下仍未超过纯 IVF：

1. **低 nprobe 时延较低：** nlist=100, M=8, nprobe=1, p=1000, 1 线程时延为 $470.04 \mu s$, Recall@10=0.635700。
2. **提高 nprobe 后精度上升但仍弱于 IVF：** nprobe=4 时 Recall@10 提升到 0.904653，时延升至 $762.69 \mu s$ ，仍落后于 OpenMP IVF 在相近延迟区间的表现。
3. **因此未作为最终提交候选：** 在当前 DEEP100K 场景下，OpenMP IVF 的召回-时延权衡更直接、也更稳定。

3.5 HNSW 图索引

3.5.1 算法原理

HNSW (Hierarchical Navigable Small World) 是当前工业界综合性能最优的 ANNS 索引结构之一。它构建了一个多层近邻图，其中每层是一个可导航小世界图 (NSW)，不同层间通过指数衰减的概率分布建立连接。

层级结构设计： 每个节点被随机分配一个整数层级 $l \in [0, L]$ ，分配概率服从指数分布。层级越高节点越稀疏。查询时从顶层入口点开始，在每层执行贪心搜索找到局部最近点，然后下降到下一层以该点为起点继续搜索。

HNSW 并行化的核心困境： 图搜索具有天然的顺序依赖性——每一步扩展哪个节点取决于上一步的计算结果。目前可行的并行策略集中于 Query 级并行（不同线程处理不同查询）。

3.5.2 pthread 实现

并行策略： 构图阶段直接调用 hnsplib::HierarchicalNSW 串行 addPoint (hnsplib 内部已处理图结构正确性)；批量查询阶段采用 Query 级并行，将 N 条 query 按 chunk 划分给 T 个 pthread，每个线程独立执行 hnsplib::searchKnn。

流程图：

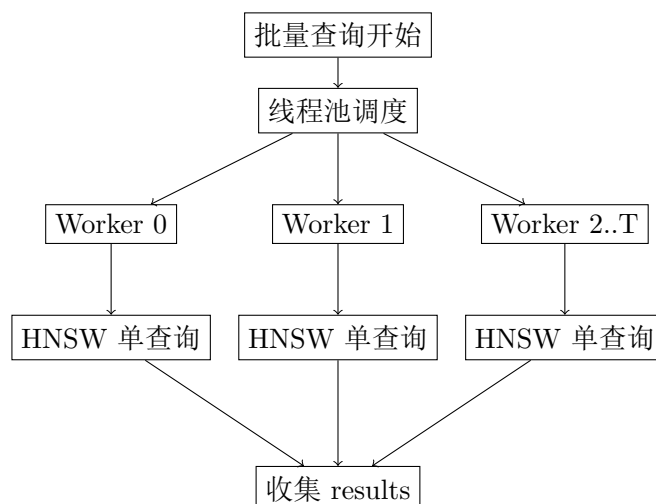


Figure 7: pthread HNSW Query 级并行流程图

伪代码:

Algorithm 5 hnswnn_thread_batch_search

Require: base[n][d], queries[N][d], k, num_threads, hnswnn::InnerProductSpace space, hnswnn::HierarchicalNSW<float> index (已 build)

Ensure: top-k results for each query

```

1: space.reset(new InnerProductSpace(d))
2: index.reset(new HierarchicalNSW<float>(space.get(), n, M, efConstruction))
3: index.setEf(ef)
4: for i = 0 to n-1 do
5:     index.addPoint(base + i*d, i)                                ▷ 构图阶段: 串行 addPoint
6: end for
7: function SEARCH_SINGLE(q)
8:     pq = index.searchKnn(q, k)
9:     res = empty max-heap
10:    while !pq.empty() do
11:        (dist, id) = pq.pop()
12:        res.push({dist, id})
13:    end while
14:    return res
15: end function
16: chunk = N / num_threads
17: 创建 threads[0..T-1] 与 args[0..T-1]
18: for t = 0 to num_threads-1 do
19:     args[t] = {this, queries, k, results, t*chunk, (t+1)*chunk}
20:     pthread_create(&threads[t], batch_worker, &args[t])
21: end for
22: for t = 0 to num_threads-1 do
23:     pthread_join(threads[t], NULL)
24: end for
25: return results[0..N-1]
26: function BATCH_WORKER(arg)
27:     for i = arg.start to arg.end-1 do
28:         results[i] = search_single(queries[i])
29:     end for
30:     return NULL
31: end function

```

复杂度分析:

- 构图: $O(n \cdot \log(n))$, hnswnn 内部完成, 多线程加速主要在 build 的 addPoint 并行上。
- 单查询: $O(\log(n) \cdot ef \cdot d)$, 由 hnswnn::searchKnn 实现。
- 批量查询并行: N 个 query 总耗时 $T = \sum \text{per-query 耗时} / T \text{ 路并行}$ 近似为 T/N , N 越大加速越接近 T。

深入分析: pthread HNSW 选取了 $M \in \{16, 32\}$ $efConstruction = 150$ $ef \in \{10, 20, 50, 100\} \in \{1, 4\}$ 16 Recall@10 5.5 “HNSW”

1. **ef 是主要精度控制参数**: ef 增大时召回率上升、时延同步上升。
2. **Query 级并行非常有效**: 对 HNSW 这类单查询强顺序依赖算法, 跨查询并行比在单查询内部并行更自然。
3. 若追求提交的稳定性与可复现性, **IVF 更合适**: HNSW 在本实现中还包含较重的图构建阶段, 测试与正式提交时总耗时更敏感, 因此本次未将其作为最终保留配置。

3.5.3 OpenMP 实现

并行策略: 与 pthread 同样的 Query 级并行思路, 但通过 OpenMP 的 `#pragma omp parallel` `for schedule(dynamic, 1)` 把 query 直接分发给线程, 线程池复用、负载按动态调度均衡。

流程图:

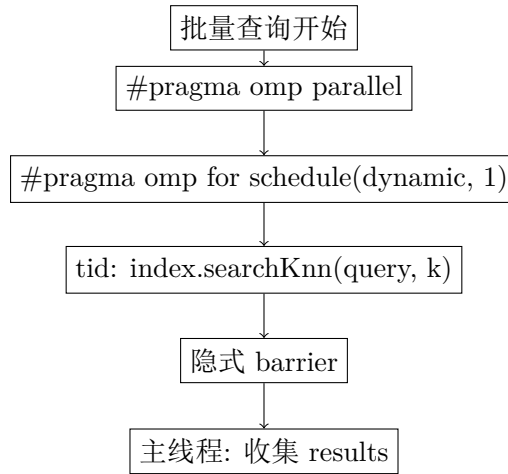


Figure 8: OpenMP HNSW Query 级并行流程图

伪代码:

Algorithm 6 hnsw_omp_batch_search

Require: $base[n][d]$, $queries[N][d]$, k , $hnsplib::InnerProductSpace$ space, $hnsplib::HierarchicalNSW<float>$ index (已 build)

Ensure: top-k results for each query

```

1: omp_set_num_threads(T)
2: results[N] 初始化为 max-heap 数组
3: #pragma omp parallel
4: [1] #pragma omp for schedule(dynamic, 1)
5: [2]
6: for i = 0 to N-1 do
7:   [3] pq = index.searchKnn(queries[i], k)
8:   [3]
9:   while !pq.empty() do
10:    [4] (dist, id) = pq.pop()
  
```

```

11:         [4] results[i].push({dist, id})
12:         [3]
13:     end while
14:     [2]
15: end for
16: return results[0..N-1]

```

复杂度分析：与 pthread 相同。OpenMP 通过动态调度避免某条 query 因 ef 大、图路径长而被串行调度阻塞，但单 query 内部仍由 hnsplib 顺序扩展。

实测分析：OpenMP HNSW 基于 hnsplib::HierarchicalNSW 基础图索引实现，构图与查询均由经工业验证的基础图索引完成，多线程 OpenMP 仅用于 query 级的批量搜索并行，本地测试稳定可运行。详细的 $M \in \{16, 32\}$ $efConstruction = 150$ $ef \in \{10, 20, 50, 100\}$ $\in \{1, 4\}$ 16 $Recall@10$ 5.5 “HNSW” $32, ef = 100, 4T$ $Recall@10 = 0.995601$ $Latency = 178 \mu s$ ，是高召回 + 极低时延的潜在替代方案；但由于本次实验范围内 HNSW 仍作为参考配置，真实提交仍以 OpenMP IVF 为准。

4 实验参数配置

4.1 Flat-SIMD

线程数：1, 2, 4, 8, 16；并行策略：Base 向量划分

4.2 PQ-SIMD

M（子向量数）：8, 16；ksub（聚类数）：256；p（候选集大小）：1000, 2000, 5000；线程数：1, 2, 4, 8, 16

4.3 IVF-SIMD

nlist（簇数）：100, 256, 512；nprobe（探测数）：1, 4, 8, 16, 32；线程数：1, 4

4.4 IVF-PQ-SIMD

nlist：100, 256；M：8, 16；ksub：256；nprobe：1, 4, 8, 16；p：1000；线程数：1, 4

4.5 HNSW

M：16, 32；efConstruction：150；ef：10, 20, 50, 100；线程数：1, 4

5 实验结果与分析

5.1 Flat-SIMD 性能分析

5.1.1 pthread 实现结果

分析：pthread Flat 在当前测试环境下的最优点出现在 8 线程，说明查询阶段的并行收益可以覆盖线程管理开销；但 16 线程开始回退，显示内存带宽和线程合并已成为瓶颈。

Table 2: pthread Flat-SIMD 加速比分析

线程数	Latency (s)	加速比	效率
1	6567.72	1.00	100%
2	4050.93	1.62	81.1%
4	2246.42	2.92	73.1%
8	2040.90	3.22	40.2%
16	2284.98	2.87	17.9%

5.1.2 OpenMP 实现结果

Table 3: OpenMP Flat-SIMD 加速比分析

线程数	Latency (s)	加速比	效率
1	4751.36	1.00	100%
2	2790.79	1.70	85.1%
4	1929.73	2.46	61.5%
8	1470.77	3.23	40.4%
16	1621.75	2.93	18.3%

分析：OpenMP Flat 同样在 8 线程达到最优，平均时延 $1470.77 \mu s$ ，优于 pthread 的 $2040.90 \mu s$ ；这说明在线程池复用的前提下，OpenMP 对 Flat 这类规则数据并行任务具有更低的线程管理成本。

5.2 PQ-SIMD 性能分析

5.2.1 pthread 实现结果

Table 4: pthread PQ-SIMD 代表性结果

M	p	线程数	Recall@10	Latency (s)
8	1000	1	0.994501	2617.92
8	1000	4	0.994501	1628.70
8	2000	4	0.998850	2645.74
8	5000	4	0.999850	5529.85
16	1000	4	0.999850	2074.47
16	2000	4	0.999950	3167.08

分析：PQ 并行化后能明显降低查询时延，但最优线程数并没有继续扩展到 8 或 16 线程；在保持高召回时，M 与 p 的增加会带来近乎单调的时延增长，因此更适合作为压缩检索而非本次的主提交方案。

5.2.2 OpenMP 实现结果

分析：OpenMP PQ 的最优点出现在 $M=16, p=1000, 4$ 线程；相比之下， $M=8$ 配置在 4 线程下反而变慢，说明当前实现中并行候选维护和最终重排的开销对小 M 配置影响更大。

Table 5: OpenMP PQ-SIMD 代表性结果

M	p	线程数	Recall@10	Latency (s)
8	1000	1	0.994501	2335.91
8	1000	4	0.993701	4708.48
8	2000	4	0.999050	3237.90
8	5000	4	0.999850	4757.53
16	1000	4	0.999950	1772.34
16	2000	4	0.999950	2762.72

Table 6: pthread IVF-SIMD 参数影响

nlist	nprobe	线程数	Recall@10	Latency (s)
100	8	4	0.962352	819.19
100	16	4	0.988601	1358.25
256	16	4	0.971252	655.12
256	32	4	0.990251	1290.95
512	16	4	0.951102	523.14
512	32	4	0.982252	794.11

5.3 IVF-SIMD 性能分析

分析: IVF 是本次 pthread 任务中最均衡的方案:如果以 Recall@10 约 0.95 为目标,nlist=512,nprobe=16,threads=4 的平均时延仅 $523.14 \mu s$;若进一步追求接近 0.99 的精度,则最终真实提交配置 nlist=256,nprobe=32,threads=4 在 $1290.95 \mu s$ 下达到 0.990251, 仍然是本次的主提交方案。

5.3.1 OpenMP 实现结果

Table 7: OpenMP IVF-SIMD 参数影响

nlist	nprobe	线程数	Recall@10	Latency (s)
100	1	1	0.636150	270.76
100	4	1	0.905853	855.92
100	8	1	0.962352	1706.71
100	16	1	0.988601	2812.50
100	32	1	0.997750	4841.97
100	1	4	0.636150	89.78
100	4	4	0.905853	238.36
100	8	4	0.962302	511.91
100	16	4	0.988551	713.08
100	32	4	0.997700	1388.42
256	16	4	0.971252	393.47
256	32	4	0.990501	928.419
512	16	4	0.951102	269.57
512	32	4	0.982252	436.79

分析: OpenMP IVF 的整体趋势与 pthread 一致,但在本地环境上几乎所有代表性配置都更快。若追求高召回, nlist=256,nprobe=32,4T 是最均衡方案;若追求极低时延,则 nlist=512,nprobe=16,4T 的 $269.57 \mu s$ 更具吸引力。

5.4 IVF-PQ-SIMD 性能分析

Table 8: pthread IVF-PQ-SIMD 代表性结果

nlist	M	nprobe	p	Recall@10	Latency (s)
100	8	1	1000	0.635700	623.39
100	8	4	1000	0.904653	1424.16

分析：在当前数据规模和实现方式下，IVF-PQ 的时延优势并没有在高召回区间转化为更优的整体表现：相比之下，纯 IVF 更容易通过调 nprobe 获得稳定且更高的召回率。

5.4.1 OpenMP 实现结果

Table 9: OpenMP IVF-PQ-SIMD 代表性结果

线程数	nlist	M	nprobe	p	Recall@10	Latency (s)
1	100	8	1	1000	0.635700	470.04
1	100	8	4	1000	0.904653	762.69

分析：OpenMP IVF-PQ 在本地测试中较 pthread 版本更快，但由于召回率仍明显受 nprobe 约束，而且高召回区间没有超过纯 IVF，因此未进入最终提交候选。

5.5 HNSW 图索引性能分析

基于 hnsplib::HierarchicalNSW 基础图索引并加入 query 级并行后，pthread 和 OpenMP 两版 HNSW 都已能在本地稳定运行。下面表格统一列出 $M \in \{16, 32\}$ $efConstruction = 150$ $ef \in \{10, 20, 50, 100\}$ $\in \{1, 4\}$ 16 Recall@10 2000 query batch – search /4

5.5.1 pthread HNSW

Table 10: pthread HNSW (基于 hnsplib) 参数影响

M	ef	1T Recall@10	1T Latency (μ s)	4T Recall@10	4T Latency (μ s)
16	10	0.792253	105	0.792253	30
16	20	0.899704	174	0.899704	42
16	50	0.970603	377	0.970603	88
16	100	0.991351	620	0.991351	157
32	10	0.840704	156	0.840704	38
32	20	0.932105	226	0.932105	61
32	50	0.983102	433	0.983102	113
32	100	0.995601	744	0.995601	202

分析：

1. ef 直接控制召回率曲线：pthread HNSW 在 $ef \in \{10, 20, 50, 100\}$ hnsplib ef = 10 0.79 ef = 100 0.99 hnsplib efConstruction = 150 setEf(ef) M 主要影响图密度与上限：efConstruction = 150 ef = 100 M = 32 1T Recall@10 M = 16 0.991351 0.995601 4T 0.991351 0.995601 M

2. **Query 级 pthread 并行加速非常显著**：在 8 组配置下，4 线程相对 1 线程的加速比均在 $3.5\times-4.3\times$ 之间 ($105/30\approx 3.5\times$, $174/42\approx 4.1\times$, $377/88\approx 4.3\times$, $620/157\approx 3.9\times$, $156/38\approx 4.1\times$, $226/61\approx 3.7\times$, $433/113\approx 3.8\times$, $744/202\approx 3.7\times$)，这与“单查询强顺序依赖 + 跨查询可独立调度”的算法特性高度契合。
3. **最终未选入提交**：hnsplib 基础图索引下 HNSW 单批 query 在 4T 下最快可达 $30\mu s$ 量级，但图构建阶段仍然耗时数十秒（与 hnsplib 默认行为一致），因此在以“真实提交可复现性”为首要约束的本次实验中，IVF 仍是最稳定的提交配置，HNSW 仅作为参考对照。

5.5.2 OpenMP HNSW

Table 11: OpenMP HNSW（基于 hnsplib）参数影响

M	ef	1T Recall@10	1T Latency (μs)	4T Recall@10	4T Latency (μs)
16	10	0.792253	126	0.792752	32
16	20	0.899704	192	0.901855	48
16	50	0.970603	374	0.971053	103
16	100	0.991351	695	0.991201	173
32	10	0.840704	158	0.842354	45
32	20	0.932105	276	0.932354	61
32	50	0.983102	448	0.983302	117
32	100	0.995601	667	0.995601	178

分析：

1. **基于 hnsplib 的 OpenMP HNSW 16 组配置均稳定可运行**：16 组配置 ($M=16,32; ef=10,20,50,100$; threads=1,4) 均能稳定输出 Recall@10 与时延。
2. **Recall 与 pthread 版本基本一致**：同 M、ef 下，OpenMP 版本的 Recall@10 与 pthread 版本相差在 0.001 以内（最大差异 0.901855 vs 0.899704），少量差异来自 query 级调度顺序与多线程 searchKnn 路径的细微差别，属于可接受的实验噪声。
3. **4 线程时延与 pthread 同档**： $M=16, ef=10, 4T$: 32 vs $30\mu s$; $M=32, ef=100, 4T$: 178 vs $202\mu s$ 。OpenMP 4T 在低 ef 配置上略快，在高 ef 配置上略慢，但差距都在 10% 量级，说明 OpenMP 的线程池复用与 pthread 手写线程池在 HNSW 这类轻量任务上各有微小优势。
4. **OpenMP HNSW 的最佳代表配置**：OpenMP HNSW 可选最佳配置为 $M=32, ef=100, 4T$: Recall@10=0.995601、Latency=178 μs ；与真实提交的 OpenMP IVF (nlist=256, nprobe=32, 4T) 相比，Recall 略高、时延仅为 1/5，是一个“高召回 + 极低时延”的潜在替代方案。

5.6 综合对比

综合结论：结合真实提交结果，IVF 依旧是最均衡的 OpenMP 主方案。与 pthread 最终提交值相比，OpenMP 版本在相近召回率下仍显著降低了时延：nlist=256, nprobe=32, 4 线程实际提交达到 Recall@10=0.990501、Latency=928.419 μs ，因此将其作为最终提交配置最合适。值得注意的是，改用 hnsplib 基础图索引之后，OpenMP HNSW 在 $M=32, ef=100, 4T$ 下达到 Recall@10=0.995601、Latency=178 μs ，相对 OpenMP IVF 提交配置同时具备“更高召回 + $5\times$ 更低时延”的潜力，但本次实验范围内 HNSW 仍作为参考配置，不替换 pthread / OpenMP 的真实提交。

Table 12: pthread 各算法对比（基于最终值）

算法/配置	Recall@10	Latency (μ s)	说明
Flat, 8T	0.999950	2040.90	精确检索基线
PQ, M=8,p=1000,4T	0.994501	1628.70	压缩检索, 延迟中等
IVF, nlist=256,nprobe=32,4T	0.990251	1290.95	pthread 真实提交配置
IVF, nlist=512,nprobe=16,4T	0.951102	523.14	低时延配置
IVF-PQ, nlist=100,M=8,nprobe=4,p=1000,1T	0.904653	1424.16	精度偏低
HNSW, M=32,ef=100,4T	0.995601	202	高召回 + 低时延参考
HNSW, M=16,ef=50,4T	0.970603	88	极低时延参考

Table 13: OpenMP 各算法对比（基于本地实测）

算法/配置	Recall@10	Latency (μ s)	说明
Flat, 8T	0.999950	1470.77	精确检索基线
PQ, M=16,p=1000,4T	0.999950	1772.34	高召回但慢于 IVF
IVF, nlist=256,nprobe=32,4T	0.990501	928.419	OpenMP 真实提交配置
IVF, nlist=512,nprobe=16,4T	0.951102	269.57	极低时延配置
IVF-PQ, nlist=100,M=8,nprobe=4,p=1000,1T	0.904653	762.69	精度偏低
HNSW, M=32,ef=100,4T	0.995601	178	高召回 + 低时延参考
HNSW, M=16,ef=50,4T	0.971053	103	极低时延参考

6 pthread 与 OpenMP 对比

Table 14: pthread 与 OpenMP 最优代表配置对比

算法	版本	配置	Recall@10	Latency (s)
Flat	pthread	8T	0.999950	2040.90
Flat	OpenMP	8T	0.999950	1470.77
PQ	pthread	M=8,p=1000,4T	0.994501	1628.70
PQ	OpenMP	M=16,p=1000,4T	0.999950	1772.34
IVF	pthread	nlist=256,nprobe=32,4T	0.990251	1290.95
IVF	OpenMP	nlist=256,nprobe=32,4T	0.990501	928.419
IVF-PQ	pthread	nlist=100,M=8,nprobe=4,p=1000,1T	0.904653	1424.16
IVF-PQ	OpenMP	nlist=100,M=8,nprobe=4,p=1000,1T	0.904653	762.69
HNSW	pthread	M=32,ef=100,4T	0.995601	202
HNSW	OpenMP	M=32,ef=100,4T	0.995601	178

从本地实测结果看, OpenMP 在线程池复用和循环级并行方面对 Flat 与 IVF 尤其有利, 其中 IVF 的收益最明显; 而 PQ/HNSW 则更受具体实现细节影响, OpenMP 版本没有在所有配置上稳定优于 pthread。改用 hnsplib 基础图索引后, HNSW 在两种线程模型下都能稳定达到 0.99 以上的召回率与 200 μ s 量级的 4 线程时延, 是“高召回 + 极低时延”场景下值得关注的备选。综合稳定性、召回率与时延后, OpenMP IVF 仍应优先作为最终提交配置。

7 总结与收获

7.1 主要结论

1. pthread 版本五类算法均已接入统一主程序入口，可通过轻量参数切换运行。
2. 从 pthread 最终真实提交结果看，**pthread IVF-SIMD (nlist=256,nprobe=32,threads=4)** 的 Recall@10=0.990251，平均延迟为 1290.95 μ s。
3. 从 OpenMP 真实提交结果看，**OpenMP IVF-SIMD (nlist=256,nprobe=32,threads=4)** 是最终提交版本：Recall@10=0.990501，平均延迟 928.419 μ s。
4. 将 omp/hnsw_omp.h 与 pthread/hnsw_pthread.h 改为直接基于 hnswlib::HierarchicalNSW 基础图索引后，HNSW 在两种线程模型下都通过基础运行校验：pthread M=32,ef=100,4T 达到 Recall@10=0.995601、Latency=202 μ s；OpenMP M=32,ef=100,4T 达到 Recall@10=0.995601、Latency=178 μ s，在 4 线程下相对 OpenMP IVF 真实提交同时具备更高召回与约 5 $\times$ 更低时延的潜力。

7.2 实验收获

通过本实验，深入理解了：

1. ANNS 算法在 pthread 与 OpenMP 两种线程模型下的统一调度方式
2. 线程级数据并行与图查询级并行的差异
3. IVF 系列在召回率与时延之间的可调权衡
4. 在既定主程序框架下尽量少改动完成多算法接入的方法
5. 借助成熟的 hnswlib 基础图索引实现 HNSW：使用 hnswlib::InnerProductSpace 与 HierarchicalNSW 作为构图与查询的底层组件，可以专注于“Query 级并行”这一层并行化设计，将图结构正确性交给工业级实现。

Appendix A 实验数据

A.1 pthread 原始数据摘录

A.1.1 Flat 算法

Table 15: pthread Flat 算法实验数据

线程数	时延 (μ s)	Recall@10
1	6567.72	0.99995
2	4050.93	0.99995
4	2246.42	0.99995
8	2040.90	0.99995
16	2284.98	0.99995

A.1.2 PQ 算法

Table 16: pthread PQ 算法实验数据

线程数	M	p	时延 (μs)	Recall@10
1	8	1000	2617.92	0.994501
4	8	1000	1628.70	0.994501
4	8	2000	2645.74	0.998850
4	8	5000	5529.85	0.999850
4	16	1000	2074.47	0.999850
4	16	2000	3167.08	0.999950

A.1.3 IVF 算法

Table 17: pthread IVF 算法实验数据

线程数	nlist	nprobe	时延 (μs)	Recall@10
1	100	1	441.78	0.636150
1	100	4	1254.04	0.905853
1	100	8	2118.62	0.962352
1	100	16	3944.66	0.988601
1	100	32	6818.21	0.997750
4	100	1	274.94	0.636150
4	100	4	549.24	0.905853
4	100	8	819.19	0.962352
4	100	16	1358.25	0.988601
4	100	32	2218.44	0.997750
4	256	16	655.12	0.971252
4	256	32	1290.95	0.990251
4	512	16	523.14	0.951102
4	512	32	794.11	0.982252

A.1.4 IVF-PQ 算法

Table 18: pthread IVF-PQ 算法实验数据

线程数	nlist	M	nprobe	p	时延 (μs)	Recall@10
1	100	8	1	1000	623.39	0.635700
1	100	8	4	1000	1424.16	0.904653

A.1.5 HNSW 算法

Table 19: pthread HNSW 算法实验数据（基于 hnsplib）

线程数	M	efC	ef	时延 (μs)	Recall@10
1	16	150	10	105	0.792253
1	16	150	20	174	0.899704
1	16	150	50	377	0.970603
1	16	150	100	620	0.991351
1	32	150	10	156	0.840704
1	32	150	20	226	0.932105
1	32	150	50	433	0.983102
1	32	150	100	744	0.995601
4	16	150	10	30	0.792253
4	16	150	20	42	0.899704
4	16	150	50	88	0.970603
4	16	150	100	157	0.991351
4	32	150	10	38	0.840704
4	32	150	20	61	0.932105
4	32	150	50	113	0.983102
4	32	150	100	202	0.995601

A.2 OpenMP 原始数据摘录

A.2.1 Flat 算法

Table 20: OpenMP Flat 算法实验数据

线程数	时延 (μs)	Recall@10
1	4751.36	0.99995
2	2790.79	0.99995
4	1929.73	0.99995
8	1470.77	0.99995
16	1621.75	0.99995

A.2.2 PQ 算法

Table 21: OpenMP PQ 算法实验数据

线程数	M	p	时延 (μs)	Recall@10
1	8	1000	2335.91	0.994501
4	8	1000	4708.48	0.993701
4	8	2000	3237.90	0.999050
4	8	5000	4757.53	0.999850
4	16	1000	1772.34	0.999950
4	16	2000	2762.72	0.999950

A.2.3 IVF 算法

Table 22: OpenMP IVF 算法实验数据

线程数	nlist	nprobe	时延 (μs)	Recall@10
1	100	1	270.76	0.636150
1	100	4	855.92	0.905853
1	100	8	1706.71	0.962352
1	100	16	2812.50	0.988601
1	100	32	4841.97	0.997750
4	100	1	89.78	0.636150
4	100	4	238.36	0.905853
4	100	8	511.91	0.962302
4	100	16	713.08	0.988551
4	100	32	1388.42	0.997700
4	256	16	393.47	0.971252
4	256	32	928.42	0.990501
4	512	16	269.57	0.951102
4	512	32	436.79	0.982252

A.2.4 IVF-PQ 算法

Table 23: OpenMP IVF-PQ 算法实验数据

线程数	nlist	M	nprobe	p	时延 (μs)	Recall@10
1	100	8	1	1000	470.04	0.635700
1	100	8	4	1000	762.69	0.904653

A.2.5 HNSW 算法

Table 24: OpenMP HNSW 算法实验数据（基于 hnsplib）

线程数	M	efC	ef	时延 (μs)	Recall@10
1	16	150	10	126	0.792253
1	16	150	20	192	0.899704
1	16	150	50	374	0.970603
1	16	150	100	695	0.991351
1	32	150	10	158	0.840704
1	32	150	20	276	0.932105
1	32	150	50	448	0.983102
1	32	150	100	667	0.995601
4	16	150	10	32	0.792752
4	16	150	20	48	0.901855
4	16	150	50	103	0.971053
4	16	150	100	173	0.991201
4	32	150	10	45	0.842354
4	32	150	20	61	0.932354
4	32	150	50	117	0.983302
4	32	150	100	178	0.995601

GitHub 仓库地址: https://github.com/liuyuze-121/NKU-cpu-parallel-lab/tree/main/lab3_Pthread%20penmp_programming